



NATIONAL UNIVERSITY
of Computer & Emerging Sciences

FAST School of Management

Semester Project

Spring 2026

Introduction to Database Systems

Submitted By:

Haniya Gul 24i-5512

Saliha Qadeer 24i-5521

Hadia Nisar 24i-5560

Instructor: Abdul Wahab Tahir

Table of Contents

Introduction	3
Problem Statement.....	3
Motivation and Real World Relevance.....	3
System Design.....	3
Relational Schema	4
Normalization.....	5
Database Implementation	5
Microsoft Access Implementation	6
Advanced Features	6
Conclusion.....	8

Introduction

The Stock Market Trading and Portfolio Management System is a relational database project designed to match real life trading in the stock market and managing a stock portfolio. This system's primary aim is to streamline and structure data on investors, brokers, stocks, transactions, and portfolio performance. In a standard stock trading setting, maintenance of such records manually can frequently lead to issues like data redundancy, data inconsistencies and problems with tracking financial activities. The purpose of this project is to address these issues by creating a centralized database system to store, retrieve, and maintain correct relationships among the various entities with accurate data.

The developed system used XAMPP as a local server manager, MySQL Workbench for performing SQL queries and designing schema, draw.io for the visualization of the database design and Microsoft Access for creating forms, reports and implementing relationships. Based on the proposal, the system design and project scope were developed.

Problem Statement

There are various entities involved in managing stock investments like investors, trading records, stocks and even brokers. If there's no efficient database system, it will be hard to manage these records and it could lead to duplicate records, missing records, and wrong reporting. Typically, in actual stock markets people trade the stocks through stock brokers, and if they don't take this relationship into consideration it doesn't reflect the way that people are actually trading. Thus, this project aimed at developing a realistic trading system for a Stock Market that involves the management of investors, assignment of brokers to investors, transactions on stocks and tracking the portfolios in a single centralized database.

Motivation and Real World Relevance

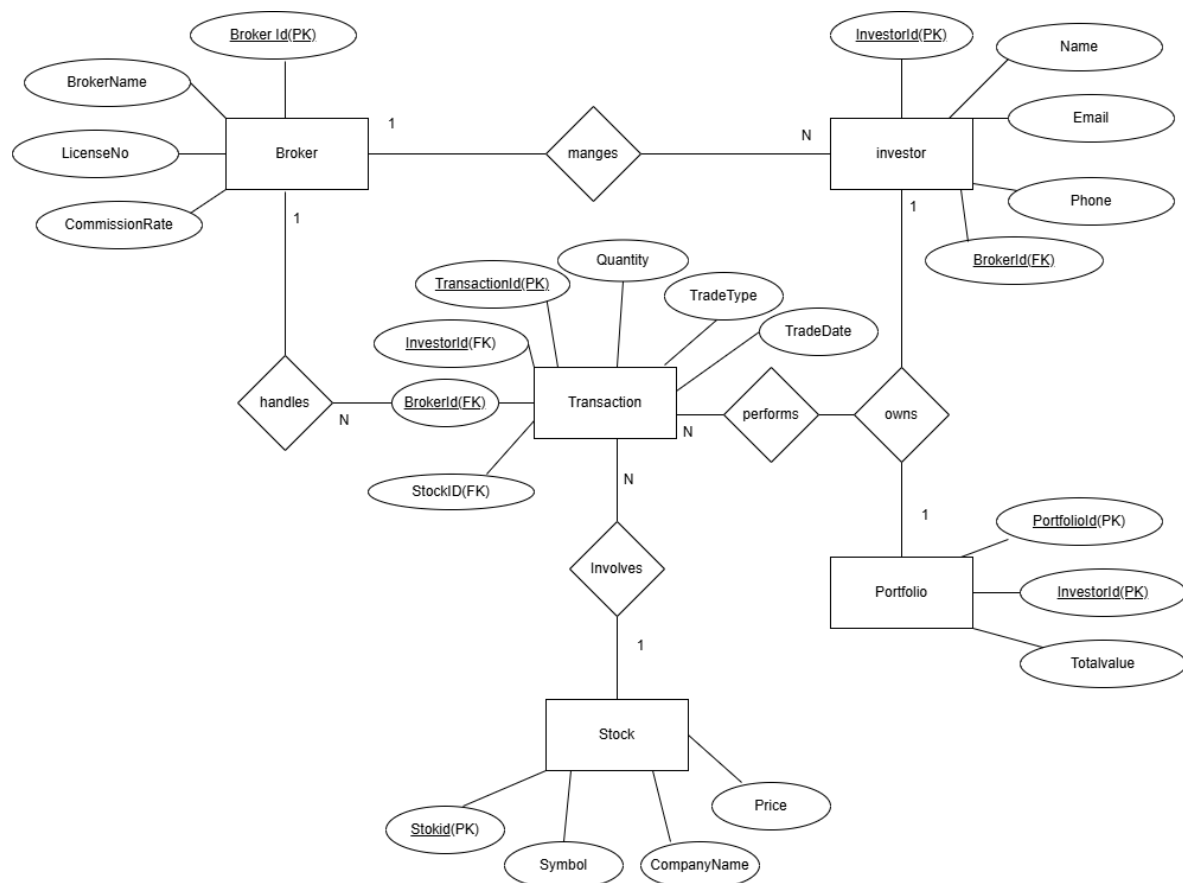
The purpose of this project was to develop a system based on the real manner in which trading takes place in the financial markets. As the involvement in stock trading has been increasing at an unprecedented level particularly in institutions like Pakistan Stock Exchange, there is a need of developing a system which can manage the investors record, brokers, stock price and trading activities efficiently. In this project, these real-life scenarios are replicated by giving an investor the ability to trade via a broker, keep a list of investment portfolios and look at the profit or loss they earn. This implies that the project is very relevant in the contemporary financial and investment context.

System Design

The five principal entities in the database system were: Broker, Investor, Stock, TransactionRecord and Portfolio. Broker entity contains all the details of brokers such as Broker names, Broker license numbers, etc. and commissions. The Investor entity contains

information about investors, such as name, email, phone number and broker assigned to them. The Stock entity is used to hold the stock symbols, company names and market prices. TransactionRecord contains all Investor's buy or sell transactions. The Portfolio entity keeps the overall value of investment held by each investor.

These relationships were modeled in accordance with the actual trading behavior in the real world. There can be one broker who operates for several investors, one investor who has several trades, one broker who has several trades, and one stock that is a part of several trades. Each investor has one and only one portfolio. The relationships were shown in E-R Diagrams and then created with Primary Keys and Foreign keys.



Relational Schema

The Broker table will be created if it does not already exist with their names and license number.

InvestorID: a number that uniquely identifies each investor. Name: The name of the individual investor.

Also created a stock table with fields like StockID which is the primary key and an integer, Symbol of the stock with the data type of text, CompanyName, and Price numeric.

The table TransactionRecord contains records of each transaction made by the Investor.

Portfolio table has PortfolioID, InvestorID and TotalValue.

Normalization

To make the database organized and avoid repeated data, normalization was applied up to Third Normal Form (3NF).

In **First Normal Form (1NF)**, each table contains single values in each field, and there are no repeated groups of data. For example, investor name, email, and phone number are stored in separate columns.

In **Second Normal Form (2NF)**, data that does not fully depend on the primary key was separated into different tables. For example, stock information such as symbol, company name, and price is stored in the Stock table instead of being repeated in transaction records.

In **Third Normal Form (3NF)**, related data was separated into different tables to remove unnecessary dependency. Broker, investor, stock, transaction, and portfolio information are stored in separate tables.

By applying normalization, the database becomes easier to manage, reduces duplicate data, and improves consistency.

Database Implementation

The project was done in MySQL Workbench using SQL commands. The schema of the database was generated by CREATE TABLE and some realistic data for the stock market was loaded into the tables by INSERT statements. Referential integrity and consistency were achieved by using primary key and foreign key constraints.

```
4 ● ○ create table Broker(
5     BrokerID int primary key auto_increment ,
6     BrokerName varchar(150) not null,
7     LicenseNo varchar(50) unique not null,
8     CommissionRate decimal(5,2) not null);
9
10 ● ○ create table Investor(
11     InvestorID int primary key auto_increment,
12     Name varchar(100) not null,
13     Email varchar(100) unique,
14     Phone varchar(20),
15     BrokerID int,
16     foreign key (BrokerID) references Broker(BrokerID)
17 );
18
19 ● ○ create table Stock(
20     StockID int primary key auto_increment,
21     Symbol varchar(20) unique not null,
22     CompanyName varchar(100) not null,
23     Price decimal(10,2) not null);
24
25 ● ○ create table TransactionRecord(
26     TransactionID int primary key auto_increment,
27     InvestorID int,
28     BrokerID int,
29     StockID int,
30     Quantity int not null,
31     TradeType varchar(10) not null,
32     TradeDate date not null,
33     foreign key (InvestorID) references Investor(InvestorID),
34     foreign key (BrokerID) references Broker(BrokerID),
35     foreign key (StockID) references Stock(StockID)
36 );
37
38 ● ○ create table Portfolio(
39     PortfolioID int primary key auto_increment,
40     InvestorID int unique,
41     TotalValue decimal(12,2),
42     foreign key (InvestorID) references Investor(InvestorID)
43 );
44
```

A number of SQL queries were added to the system to show the functionality of the system. The queries involved total investment per investor, profit or loss per stock, total earnings from broker commission, most active broker (using COUNT), and monthly transaction summaries. Throughout the implementation JOIN operations, aggregate functions, grouping

and sorting were used. Moreover, InvestorID and StockID were added to be indexed for better query performance.

In addition to standard SQL queries, stored procedures were also implemented to improve modularity and reusability of database operations. A procedure named *ShowAllBrokers* was created to retrieve broker records efficiently, while another procedure named *GetInvestorPortfolio* was implemented to fetch portfolio details of a specific investor. These procedures help simplify repetitive database operations and improve the overall organization of SQL logic.

Microsoft Access Implementation

The data in the database was also exported to Microsoft Access for the project requirement of forms and reports. Five tables were successfully imported into Access: Broker, Investor, Stock, TransactionRecord and Portfolio. Relationships were established between these tables to ensure consistency of information between related tables.

Two forms were created in Access to interact with users. The InvestorForm was constructed to manipulate Investor and Broker record; and the TransactionForm was created to document stock trading transactions containing Investor, Broker, Stock, Quantity, and Trade data.

Two queries have been developed to provide reporting and analysis. The PortfolioQuery was written so it would return the names of investors and portfolio values, and the BrokerPortfolioQuery was developed to return names of investors, the values of their portfolios, and information about their broker.

Two reports in Access were created using the two queries. There are two reports, the PortfolioReport is for investors and groups investors by their assigned brokers and the BrokerPortfolioReport groups investors by their broker and totals the portfolio values using aggregation functions. This report meets the project requirements of grouping, sorting and aggregation.

Advanced Features

A number of enhancements were made to the system. A SQL view was created for 'Portfolio Summaries' to provide an easier to report and more readable view. A special technique of indexing was used for significant attributes to speed up queries. The stock market inspired data was created from the Pakistan Stock Exchange (PSX) website to make the system realistic. An easy-to-use dashboard and reporting system were also developed in Access to improve presentation and usability.

1.

```
272
273 -- View For Portfolio Summary
274 • create view PortfolioSummary as
275   select i.Name,p.TotalValue
276   from Investor i
277   join Portfolio p on i.InvestorID=p.InvestorID;
278
279 • SELECT * FROM PortfolioSummary;
280
```

Result Grid		Filter Rows:	Export:
Name	TotalValue		
Elaf Muneeb	118265.00		
Mahnoor Javed	35613.00		
Abeer Usman	30240.00		
Zavian Raza	8040.00		
Eshal Tariq	89425.00		
Mikaal Shahmeer	52925.00		

PortfolioSummary 7 x

2.

```
281 -- Indexing
282 • create index idx_InvestorID on TransactionRecord(InvestorID);
283
284 • create index idx_StockID on TransactionRecord(StockID);
285
286 -- Procedures
```

3.

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
BrokerID	BrokerName	LicenseNo	CommissionRate	
1	AKD Securities Limited	AKD001	0.15	
2	JS Global Capital Limited	JSG001	0.20	
3	Mohammad Munir Mohammad Ahmed Khanani S...	MMK001	0.18	
4	KTrade Securities Limited	KTR001	0.15	
5	Arif Habib Limited	AHL001	0.22	
6	BMA Capital Management Limited	BMA001	0.20	

4.

```
285 DELIMITER //
```

```
286 ● CREATE PROCEDURE GetInvestorPortfolio(IN investor_id INT)
```

```
287 ○ BEGIN
```

```
288     SELECT i.Name, p.TotalValue
```

```
289     FROM Investor i
```

```
290     JOIN Portfolio p ON i.InvestorID = p.InvestorID
```

```
291     WHERE i.InvestorID = investor_id;
```

```
292 END //
```

```
293 DELIMITER ;
```

```
294 ● CALL GetInvestorPortfolio(1);
```

```
---
```

Result Grid | Filter Rows: | Export: | Wrap Cell

	Name	TotalValue
▶	Elaf Muneeb	118265.00

Conclusion

It is a successful real world implementation of a relational database system for trading and managing stocks, and a good demonstration of the design and implementation of a system. The project's features include a single platform that connects investors, brokers, stocks, transactions, and portfolio analysis, providing efficient data storage, accurate reporting, and realistic financial tracking. The use of SQL queries, database relations, forms, reports and more complex features enabled the project to satisfy all academic requirements in addition to meeting real world financial requirements.